

A Comparative Report on the Loop Scheduling Options in OpenMP

Exam No. B024703

October 27, 2016

1 Introduction

This report details experiments undertaken to compare the performance of the loop scheduling options available in OpenMP. A serial code involving two separate loops was provided to the Author. This code measured and reported the time taken for each loop to execute 1000 times.

The first loop (loop 1) iterates through the elements of a two dimensional array, calculating the cosine of each element and assigning this value to another similarly sized array. The second loop (loop 2) performs more operations per element in a similarly sized array to those in loop 1. This time a sum of varying length is performed for every element in the two dimensional array, with each term in the sum requiring the calculation of a logarithm.

The nature of these loops led to different performance characteristics under parallelisation for each. This is due to the varying complexities of calculation within and between each of these loops. The variation in amounts of operations performed per loop cycle can cause issues with load balancing, where some threads complete all assigned work before others, leading to idling and sub-optimal performance.

The purpose of the loop scheduling options provided by OpenMP is to provide a range of strategies to improve load balancing between different threads. The remainder of this report will describe the methods used to measure the effectiveness of each of the loop schedules before detailing the results of these

experiments. Finally, this report will present a conclusion drawn from these results.

2 Methods

2.1 Compiling and Running Serial Code

As a precaution, to check the correctness of any subsequent parallel version, the original code was compiled in its serial form. This was done using the Intel C Compiler with the `-O3` option specified at compile time for maximum serial optimisation. Beyond using this to verify the correctness of parallel solution and that parallel performance was credible, analysis of the performance of this serial code falls outside the scope of this report.

2.2 Parallelising the Code

Before different scheduling clauses could be used, the code had first to be parallelised. This was done using OpenMP directives and apart from these directives involved no alterations to the original source code. Within the directive for launching a parallel for loop, OpenMP gives the option of setting a schedule. The available schedule options are as follows:

- `static`
- `auto`
- `static,n`
- `dynamic,n`
- `guided,n`

Where `n` specifies a chunksize. The Intel documentation [1] describes how the programmer has the option to set the schedule using an environment variable. This was done so that the schedule could be changed programmatically and that all experiments would be easily repeatable.

2.3 Compiling and Running the Code

As with the serial case the code was compiled using the Intel C Compiler and using the `-O3` optimisation flag. In this case, however, the appropriate compiler flag was used to compile an OpenMP program. To measure the performance of the different scheduling options the code was initially run on 4 threads on a backend node of `cirrus`, an Intel Xeon powered cluster. For the schedules where it is possible to set a chunksize, this chunksize took the values 1, 2, 4, 8, 32 and 64.

From these results, the scheduling option with the best performance for each loop on 4 threads was determined. Using these two options, the best for loop 1 and the best for loop 2, the code was then run again on 1, 2, 4, 6, 8, 12 and 16 threads. This allowed the measurement of the speedup, as defined by T_p/T_1 , so that the scalability of the best performing schedule on 4 threads could be analysed.

For all measurements the running time was measured multiple times and a mean of these values used. In addition the standard deviation was also calculated. This allows the comparison of performance variability between different schedules. Performance variability is often an important consideration in HPC applications, where it may be desirable to avoid methods that often, but unreliably, yield high performance.

3 Results

3.1 Results on 4 Threads

The first results gathered were execution times for each loop for every available loop schedule.

4 conclusion

The best way to do it was:

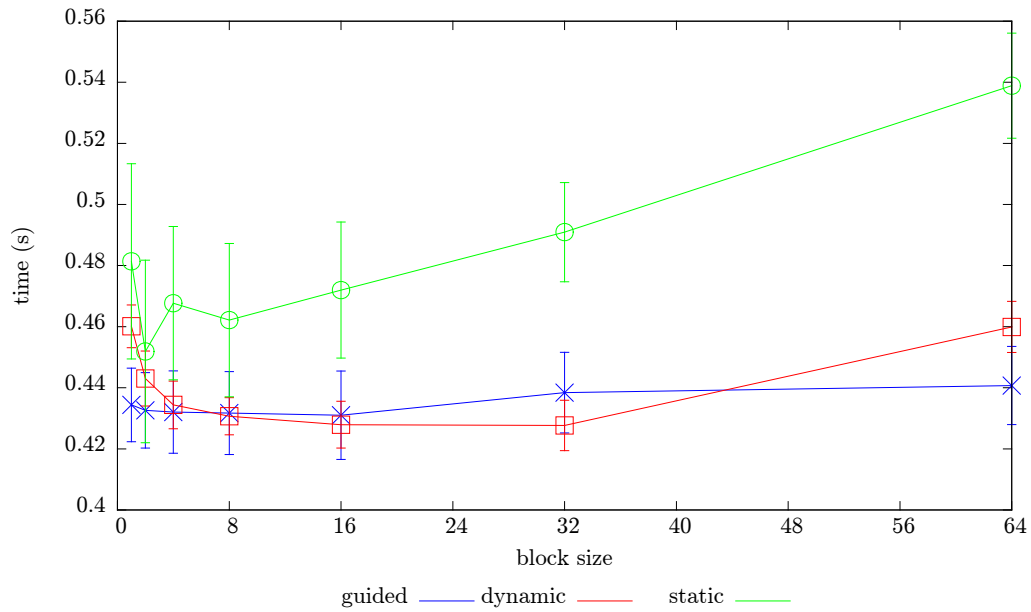


Figure 1: Graph showing execution time for loop 1 against block size for the guided, dynamic and static schedules. The error bars show 1 standard deviation. Note that these error bars are much larger than in loop 2, indicating greater performance variability.

References

- [1] Intel Corp. *OpenMP Loop Scheduling* <https://software.intel.com/en-us/articles/openmp-loop-scheduling> Accessed: 26-10-2016